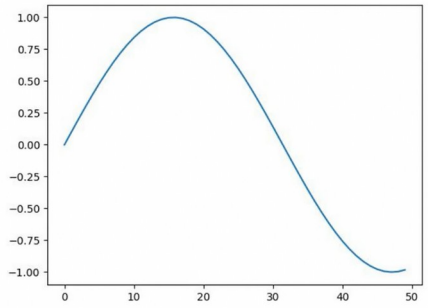
Masked Diffusion Language Models

(Simple and Effective)

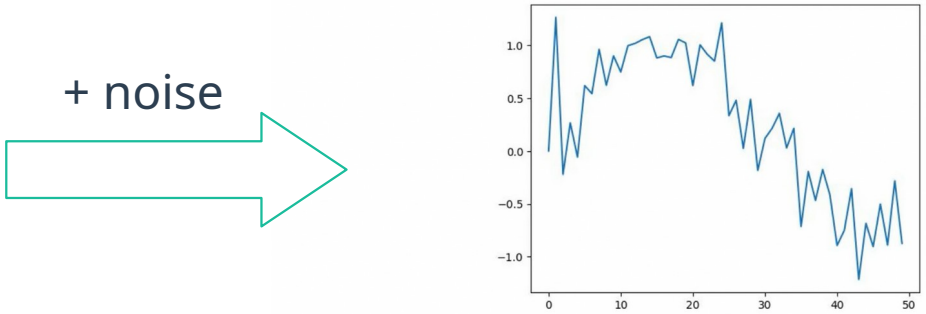
Diffusion?

$x \sim p_\theta(x)$

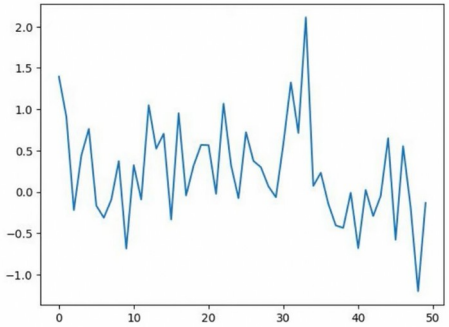




+ noise



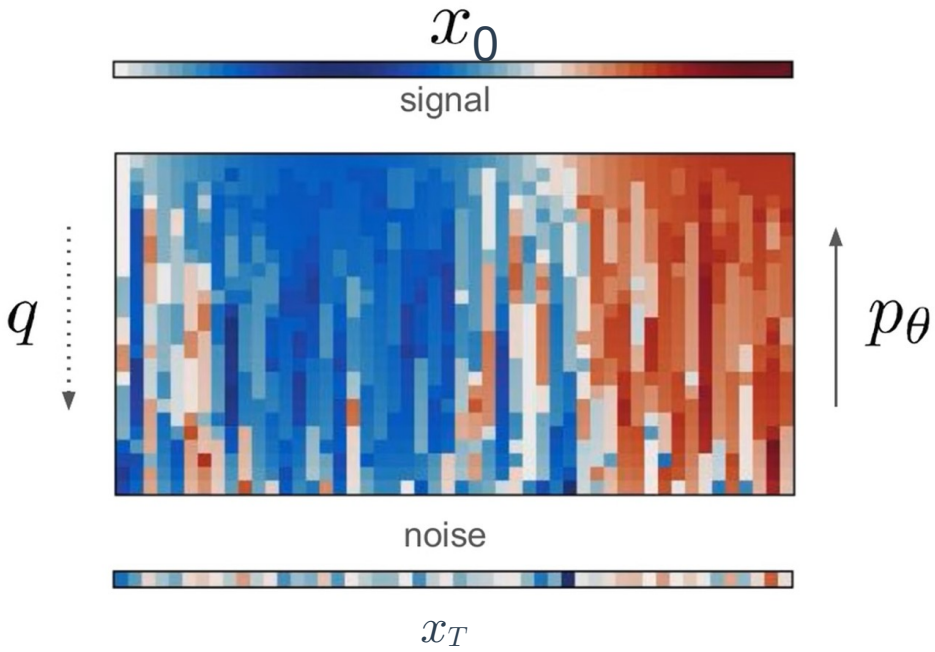






+ noise

Diffusion! (Sohl-Dickstein et al., 2015)



- **continuous (real-valued) data**
- ***forward noising process:***

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

- **posteriors (usually Gaussian):**

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

- **with *variance schedule*:**

$$\{\beta_t \in (0,1)\}_{t=1}^{T}$$

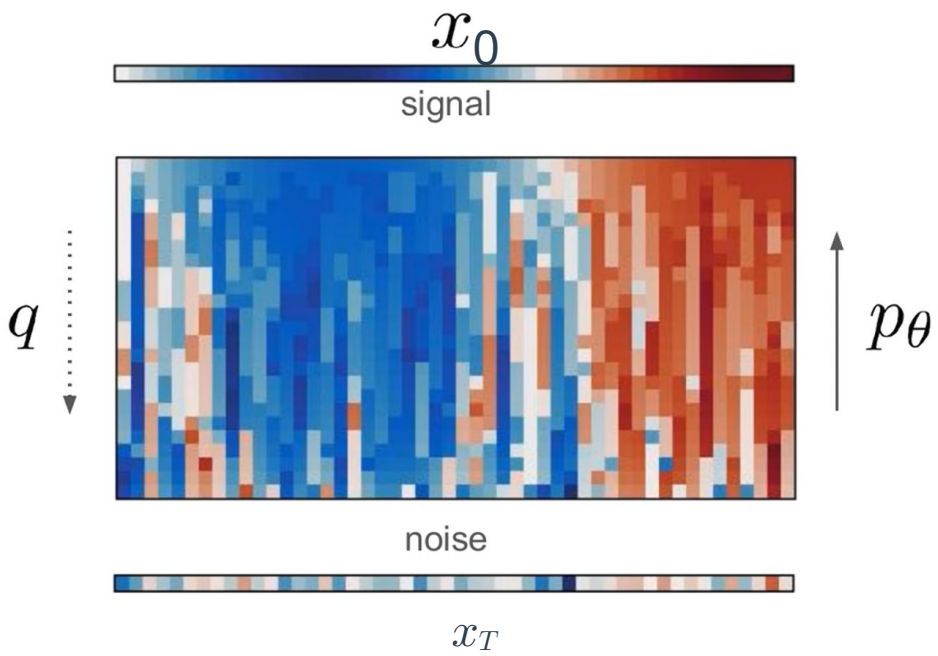
Reparameterization trick

Let, for each $t \in \{1, \dots, T\}$, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$, and $\epsilon, \epsilon', \dots \sim \mathcal{N}(0, I)$. Then

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t} x_{t-1} + \sqrt{\beta_t}\epsilon \\
&= \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\epsilon \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon') + \sqrt{1-\alpha_t}\epsilon \\
&= \sqrt{\alpha_t\alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\epsilon' + \sqrt{(1-\alpha_t)}\epsilon \\
&= \sqrt{\alpha_t\alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t)}\epsilon'' \\
&= \sqrt{\alpha_t\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\epsilon'' \\
&= \dots \\
&= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon'''
\end{aligned}
$$

Thus, $q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t) I)$.

Diffusion! (Sohl-Dickstein et al., 2015)



- **direct noising:**
  $$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$
- **goal: opposite direction, but no closed form for** $q(x_{t-1} \mid x_t)$
- **solution: learn parameterized**
  $$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t)$$
  based on Gaussian posteriors
  $$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Training diffusion models

- **Goal: maximize log-likelihood of dataset…**
  but: computing $p_\theta(x_0)$ is intractable
- **Solution: negative evidential lower bound (ELBO):**

$$L_{\text{VLB}} = \mathbb{E}_{q(x_{0:T})}\left[\log \frac{q(x_{1:T} \mid x_0)}{p_\theta(x_{0:T})}\right] \geq -\mathbb{E}_{q(x_0)} \log p_\theta(x_0)$$

- **make tractable:**

$$= \mathbb{E}_q[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}]$$

Making the reverse posteriors Gaussian

- **We have sneakily conditioned them on x_0 :**

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \mu_q(x_t, x_0), \bar{\beta}_t I)$$

- **this is a Gaussian with**

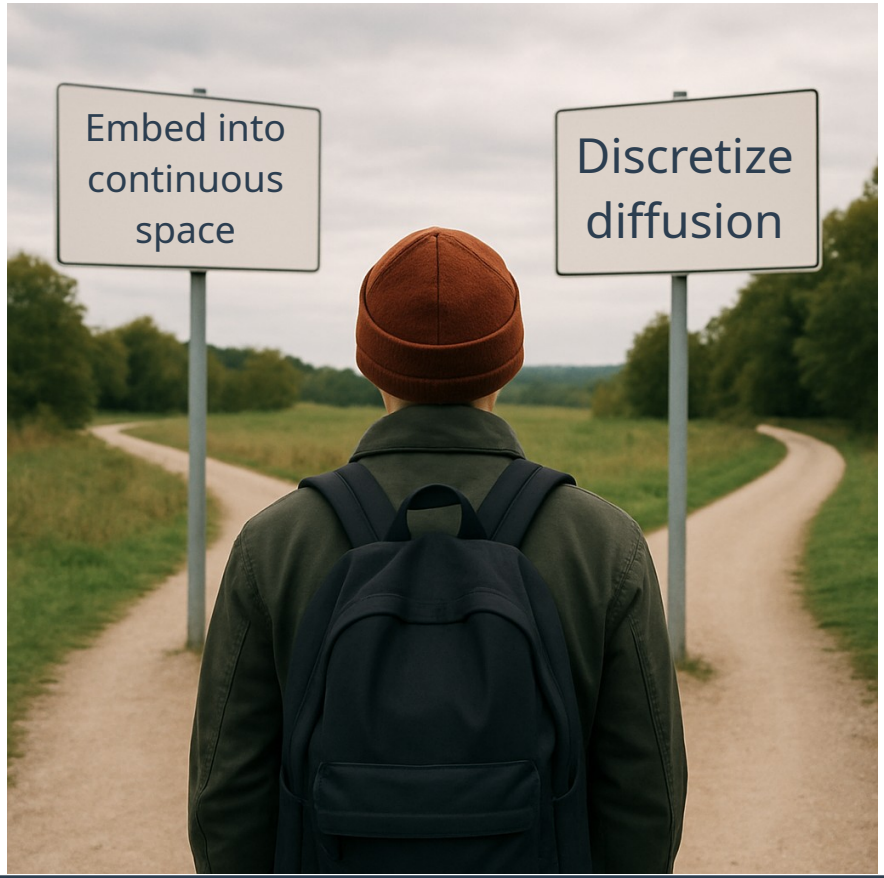
$$\mu_q(x_t, x_0) = \sqrt{\alpha_{t-1}}\beta_t/(1-\bar{\alpha}_t)x_0 + \sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})/(1-\bar{\alpha}_t)x_t$$
$$\bar{\beta}_t = (1-\bar{\alpha}_{t-1})/(1-\bar{\alpha}_t)\beta_t$$

- **obtained by Bayes' theorem and tedious calculations**

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \quad = \cdots$$


Images are nice and continuous, but text is discrete!

How shall we overcome this discrepancy?



D3PM (Austin et al., 2021)

- **fix categories** $1, \ldots, K$
- **diffusion = change of category** $(Q_t)_{ij} = q(x_t{=}j \mid x_{t-1}{=}i)$

  $$q(\vec{x}_t \mid \vec{x}_{t-1}) = \mathrm{Cat}(\vec{x}_t; \vec{p} = \vec{x}_{t-1} Q_t)$$

  (with one-hot encoding)

  $$\left[\text{He} \mathrel{\hat{=}} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \right]^{\mathrm{T}} \cdot \begin{pmatrix} 0.5 & 0.2 & 0.3 \\ 0.2 & 0.25 & 0.55 \\ 0.37 & 0.14 & 0.49 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.2 \\ 0.37 \end{pmatrix}^{\mathrm{T}}$$

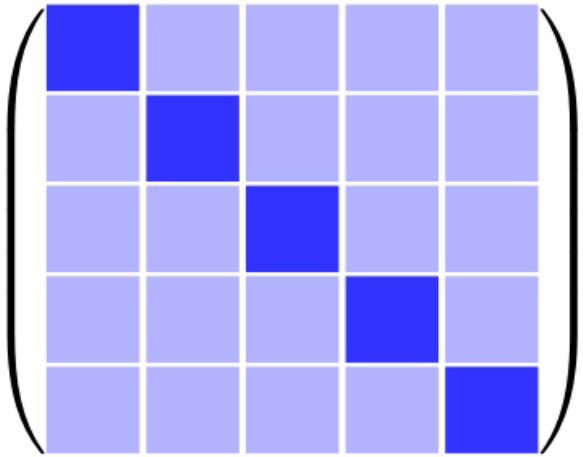
- **reparameterization works again** **(reverse omitted)**

  $$q(\vec{x}_t \mid \vec{x}_0) = \mathrm{Cat}(\vec{x}_t; \vec{p} = \vec{x}_0 \bar{Q}_t) \quad \textbf{with} \quad \bar{Q}_t = Q_1 Q_2 \cdots Q_t$$

How to transition?

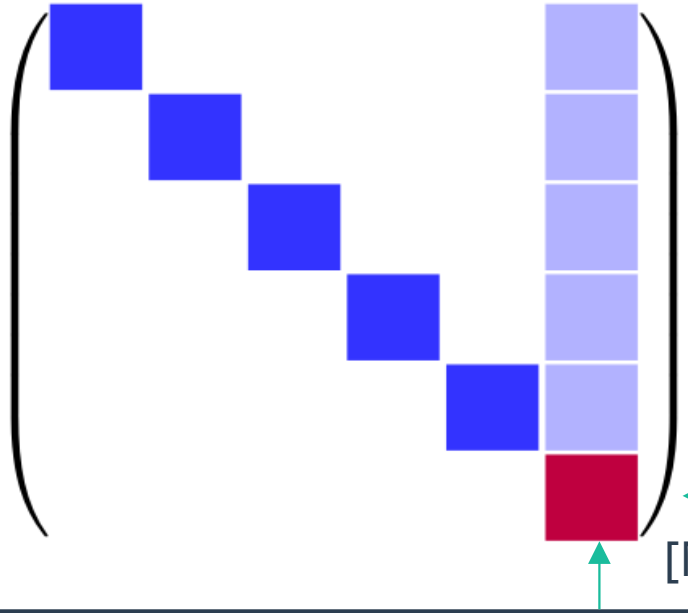
- **Uniform**

$$Q_t = (1 - \beta_t)I + \beta_t \mathbb{1}\mathbb{1}^{\mathrm{T}}/K$$



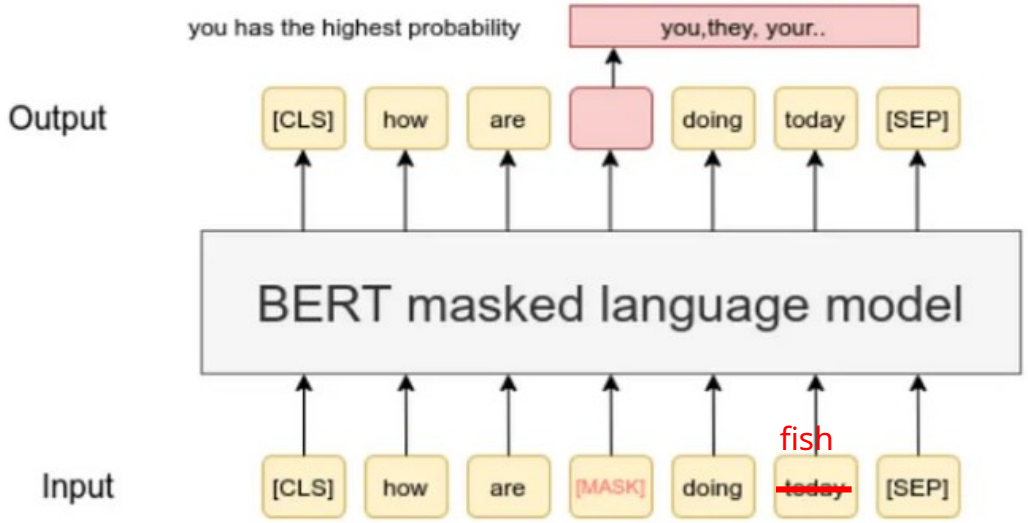
- **Absorbing state**

$$Q_t = (1 - \beta_t)I + \beta_t \mathbb{1}\vec{m}^{\mathrm{T}}$$



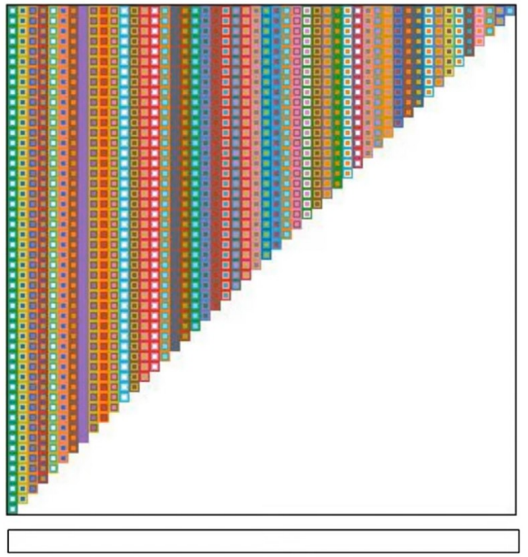
Familiar objectives

- **BERT**



1 step of uniform + absorbing state

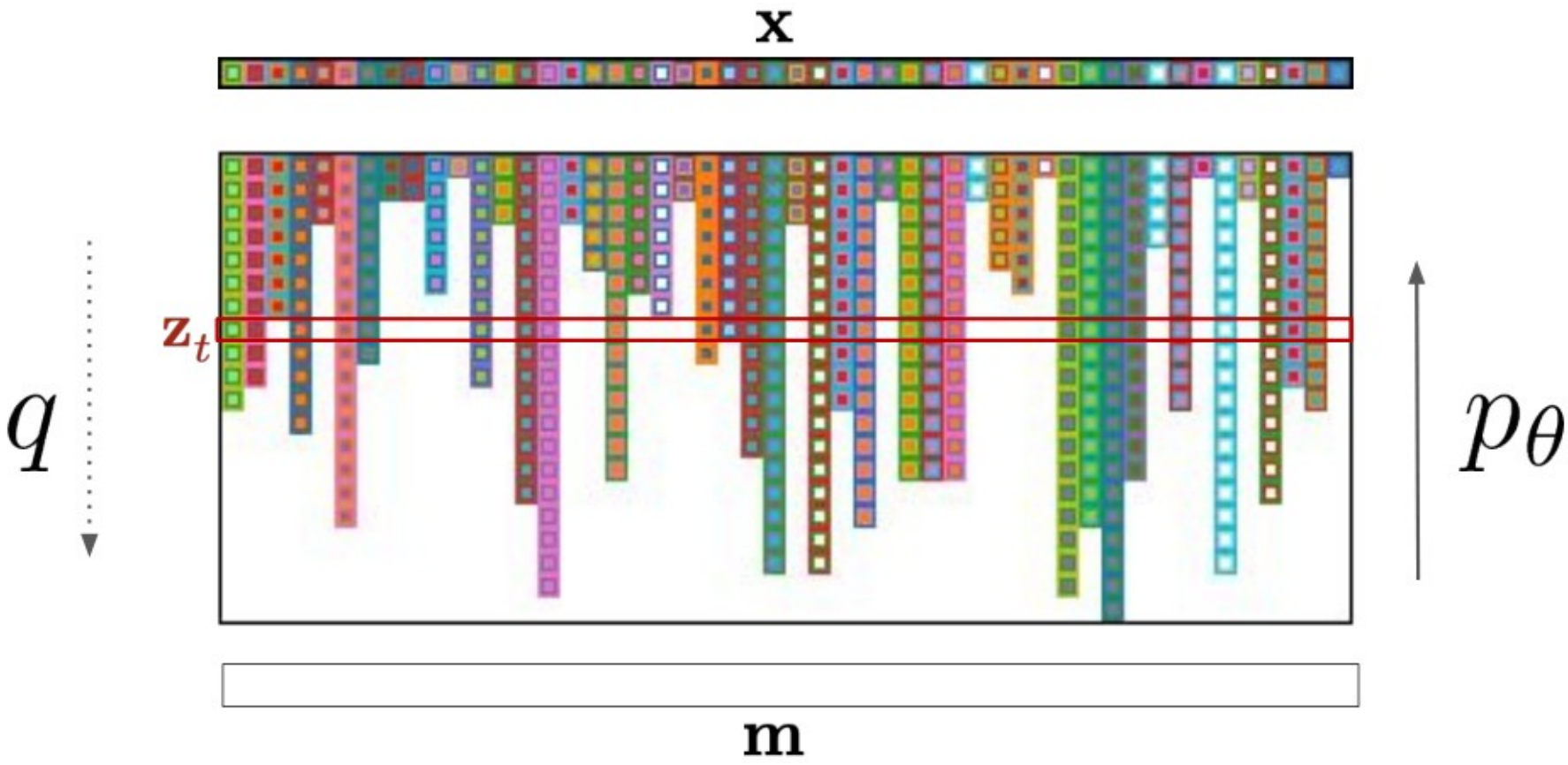
- **Autoregressive LMs**



p_θ

~ absorbing state

Simple masked diffusion language models



States: continuous → discrete?

Time: discrete → continuous!

Continuous time Markov chains w/o the theory

- **now $t \in [0,1]$ and $\alpha_t \in [0,1]$ is a strictly decreasing function in t such that $\alpha_0 \approx 1$ and $\alpha_1 \approx 0$**

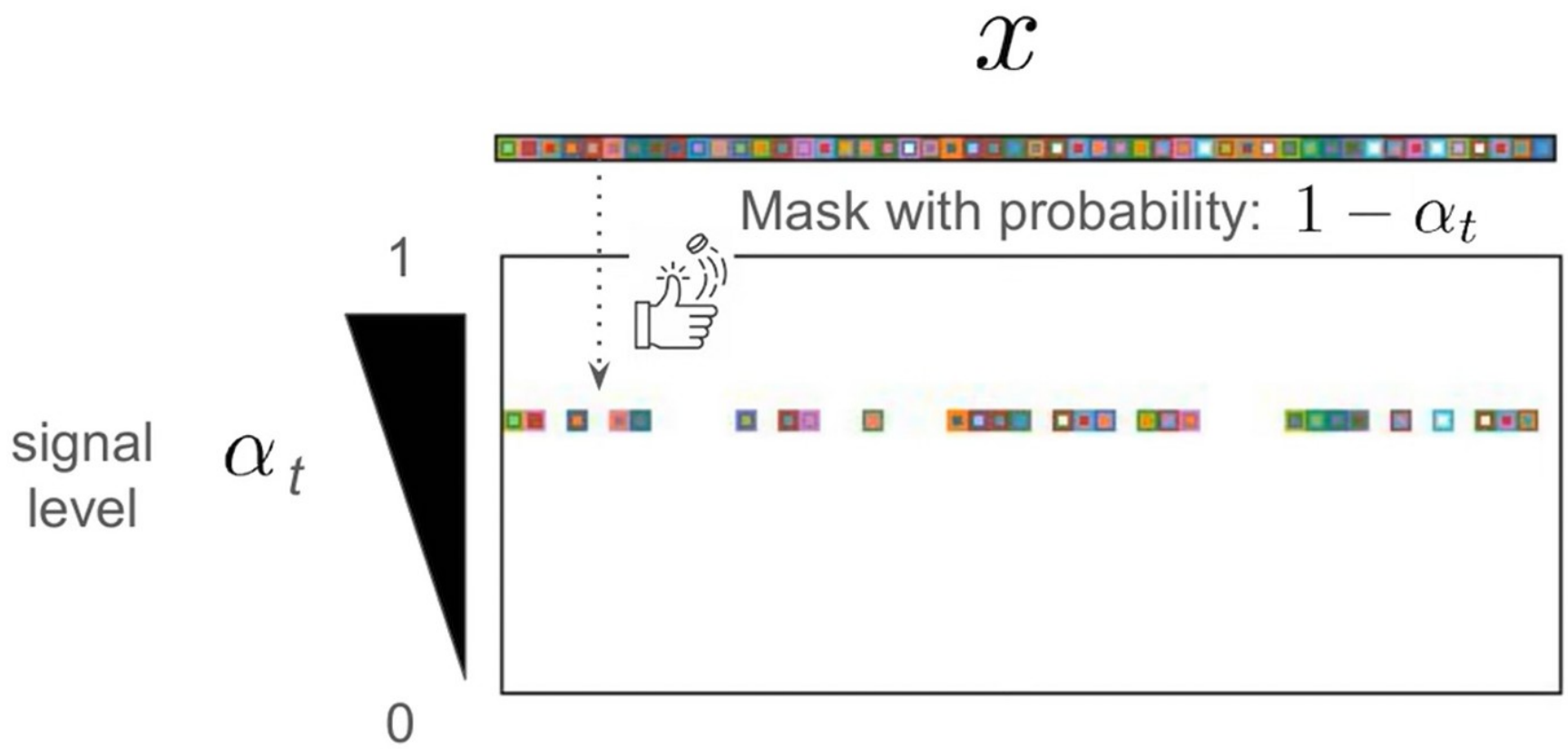
$$q(\vec{z}_t \mid \vec{z}_s) = \mathrm{Cat}(\vec{z}_t; \alpha_{t|s}\vec{z}_s + (1 - \alpha_{t|s})\vec{m}) \qquad \text{where } \alpha_{t|s} = \alpha_t / \alpha_s$$

- **we get reparameterization for free:**

$$q(\vec{z}_t \mid \vec{x}) = \mathrm{Cat}(\vec{z}_t; \alpha_t\vec{x} + (1 - \alpha_t)\vec{m})$$

- **this simply interpolates between token and mask!**

It’s really interpolating discrete diffusion



SUBS parameterization of the reverse process

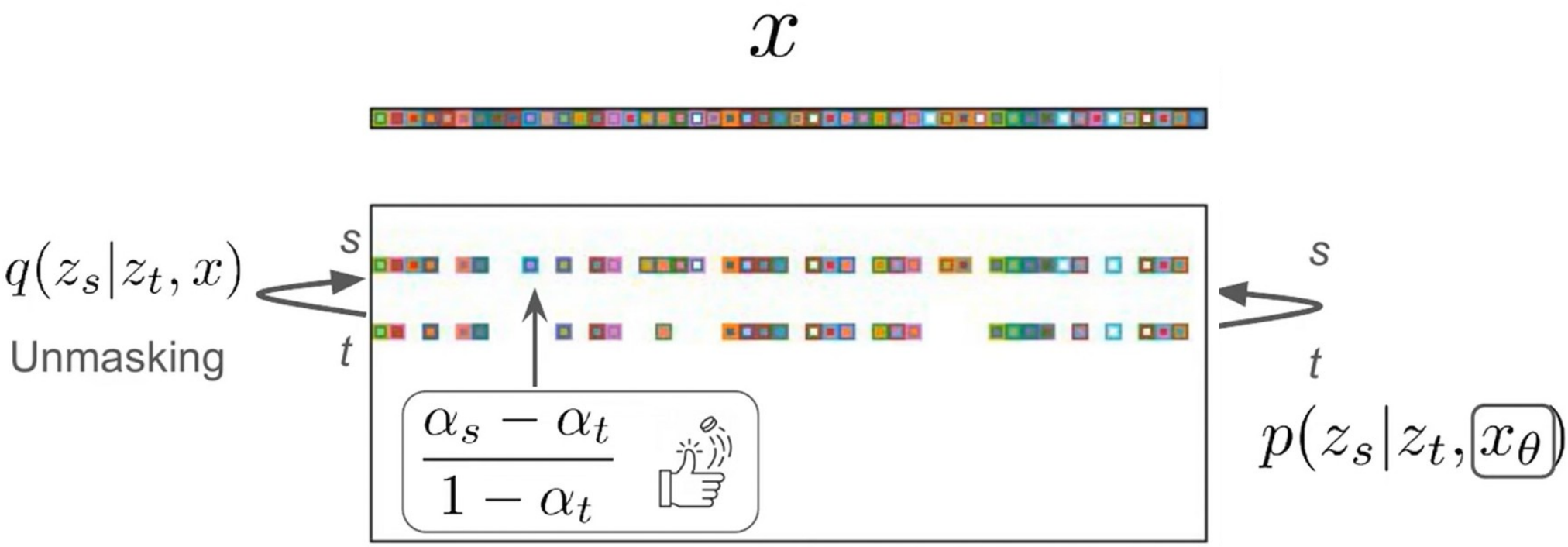
- **parameterized reverse posteriors**

$$p_\theta(\vec{z}_s \mid \vec{z}_t) = q(\vec{z}_s \mid \vec{z}_t, \vec{x} = \vec{x}_\theta(\vec{z}_t, t))$$

$$= \begin{cases} \mathrm{Cat}(\vec{z}_s; \vec{z}_t) & \vec{z}_t \neq \vec{m} \\ \mathrm{Cat}\left(\vec{z}_s; \frac{(1-\alpha_s)\vec{m} + (\alpha_s - \alpha_t)\vec{x}_\theta(\vec{z}_t, t)}{1-\alpha_t}\right) & \vec{z}_t = \vec{m} \end{cases}$$

- $\vec{x}_\theta(\vec{z}_t, t)$ **is a (neural) model, diffusion "steps" = interpolation**
- **zero masking probabilities: make** $\langle \vec{x}_\theta(\vec{z}_t, t), \vec{m} \rangle$ **zero**
- **carry-over unmasking: copy unmasked inputs to output**

Training process



Simple and effective training objectives

- **continuous NELBO:**

$$\mathcal{L}^{\infty}_{\text{NELBO}} = \mathbb{E}_{\vec{x}\sim q(\vec{x}),t\sim\mathcal{U}(0,1),\vec{z}_t\sim q(\vec{z}_t|\vec{x})} \int_{t=0}^{t=1} \frac{\alpha'_t}{1-\alpha_t} \log\langle \vec{x_\theta}(\vec{z}_t,t),\vec{x}\rangle \,\mathrm{d}t$$

- **invariant to the choice of α_t!**
- **extension to sequence data:**

$$p_\theta(\vec{z}_s^{1:L} \mid \vec{z}_t^{1:L}) = \prod_{l=1}^{L} p_\theta(\vec{z}_s^{L} \mid \vec{z}_t^{1:L})$$

$$\mathcal{L}^{\infty}_{\text{NELBO}} = \mathbb{E}_q \int_{t=0}^{t=1} \frac{\alpha'_t}{1-\alpha_t} \sum_{l=1}^{L} \log\langle \vec{x_\theta}^{\,l}(\vec{z}_t,t),\vec{x}\rangle dt$$

Training algorithm

Algorithm 1 Training MDLM

1: **repeat**

2: $\mathbf{x}^{1:L} \sim q(\mathbf{x})$ ▷ Sample a sentence.

3: $t \sim \mathcal{U}[0,1]$ ▷ Sample a time step.

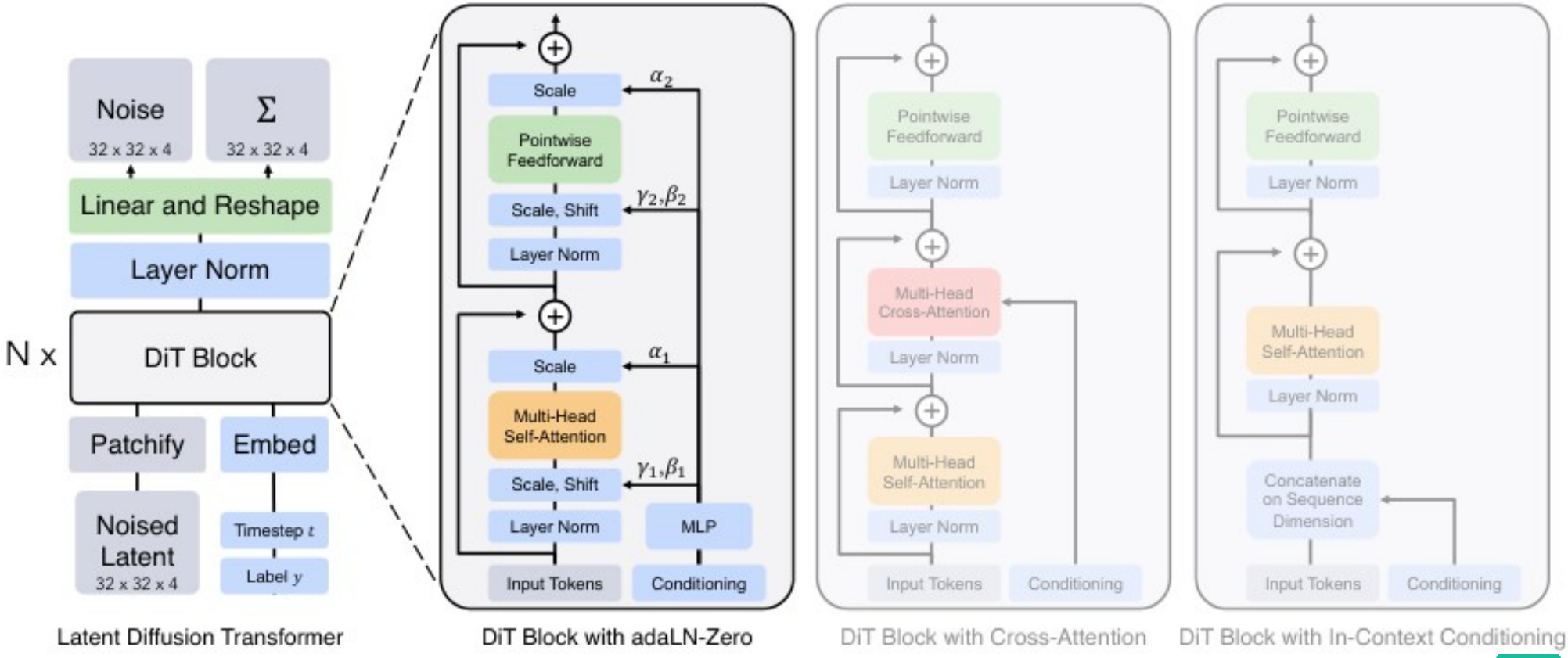
4: $\mathbf{z}_t^\ell \sim \text{Cat}(\mathbf{z}_t^\ell; \alpha_t \mathbf{x}^\ell + (1-\alpha_t)\mathbf{m}) \; \forall \, 1 \le \ell \le L$ ▷ Mask Each token $\mathbf{x}^\ell$ independently to obtain the latent $\mathbf{z}_t^{1:L}$.

5: Take gradient descent step on

$$\nabla_\theta \frac{\alpha_t'}{1-\alpha_t} \sum_\ell \log \langle \mathbf{x}_\theta^\ell(\mathbf{z}_t^{1:L}, t), \mathbf{x}^\ell \rangle$$

6: **until** converged

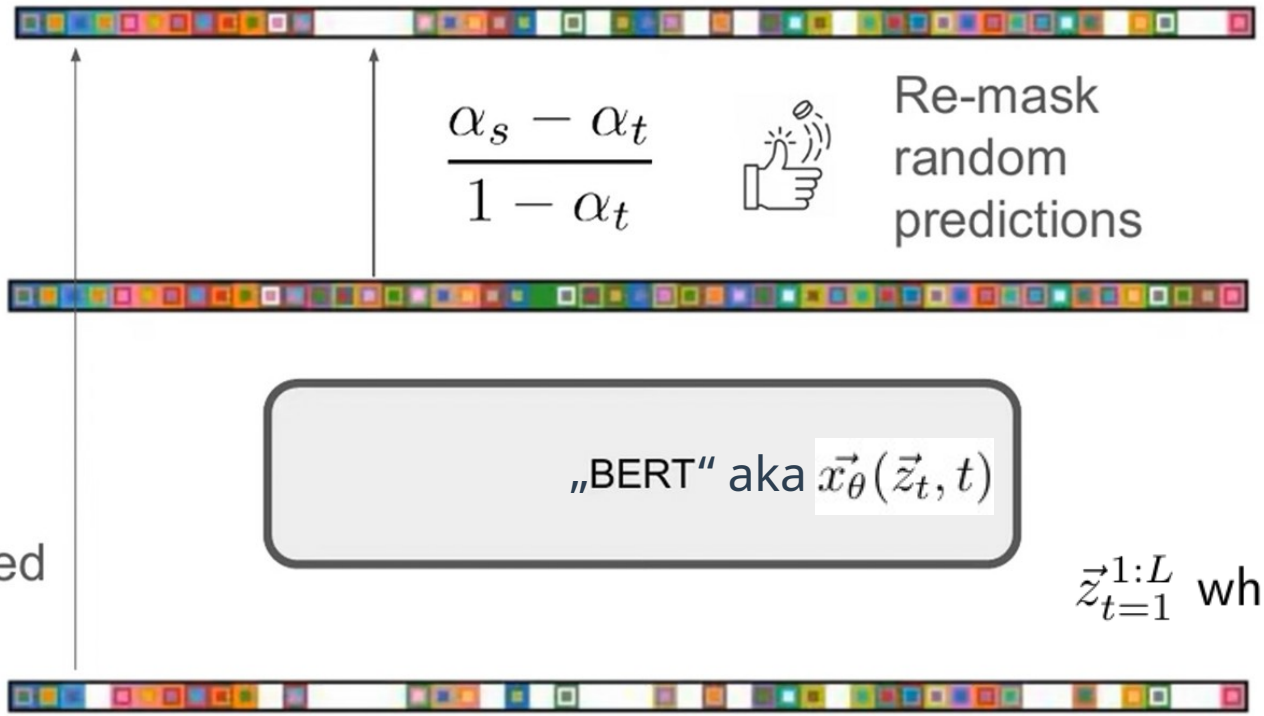
Predicting clean data: Diffusion Transformers



Inference

in multiple (discrete) steps: sample $\vec{z}_s^{\,l} \sim p_\theta(\vec{z}_s^{\,l} \mid \vec{z}_t^{\,1:L})$ for each l

from $t{=}1$ to $t{=}0$



Unbounded sequence length: semi-AR inference

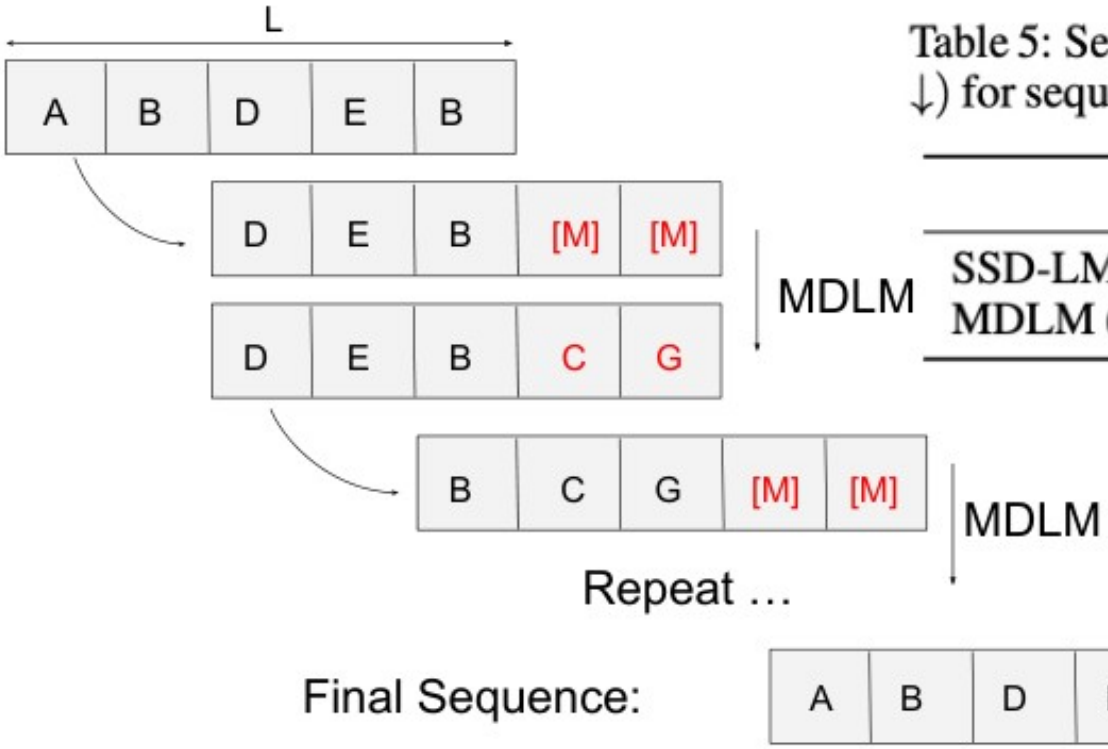
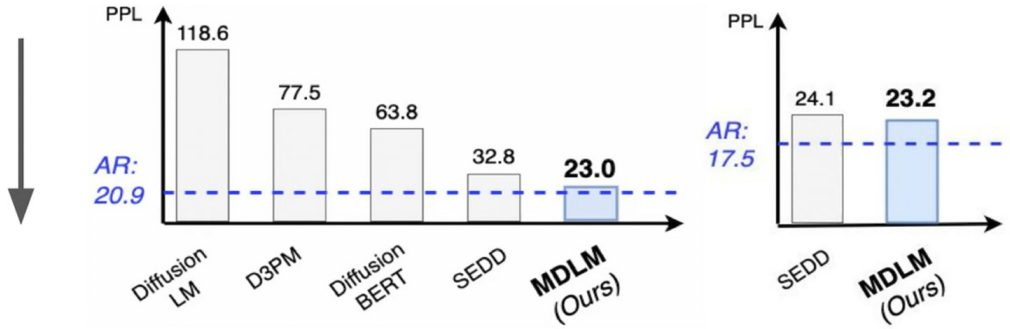


Table 5: Semi-AR generative perplexity (Gen. PPL; ↓) for sequences of 2048 tokens.

	Gen. PPL (↓)	Sec/Seq (↓)
SSD-LM	35.43	2473.9
MDLM (Ours)	**27.18**	**89.3**

Result: diffusion becomes competitive



LM1B

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

OpenWebText

Table 8: Test perplexities (PPL; ↓) for MDLM ablations on LM1B. For the discrete-time models, we use $T = 1000$. Standard deviation is measured over 5 seeds during evaluation.

	PPL (≤)
MDLM (47)	**27.04**±.01
w/o continuous time (43)	27.19±.07
& w/o carry-over (41)	28.56±.15
& w/o zero masking (39)	28.51±.15

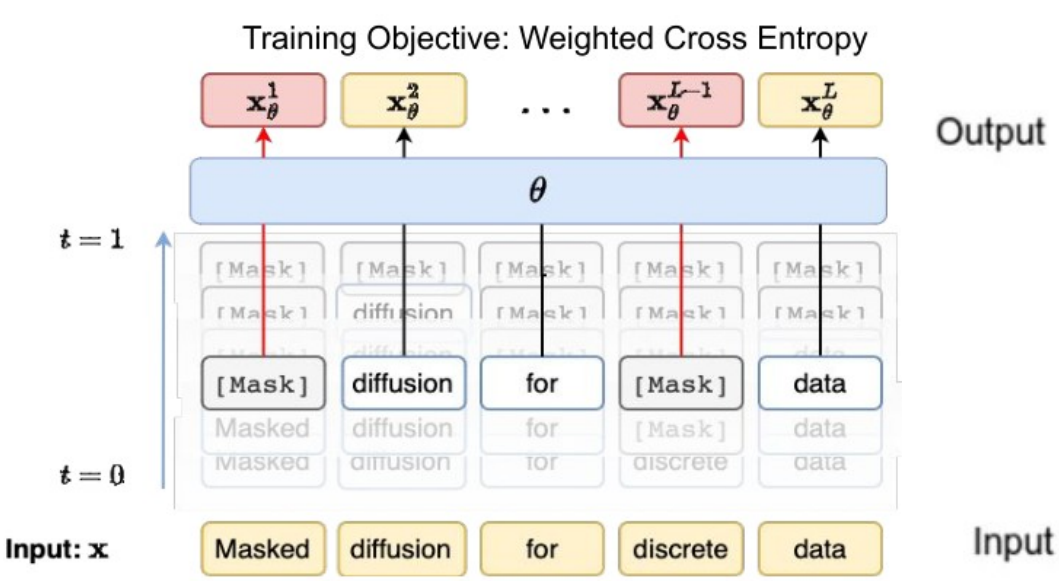
Table 3: Zero-shot perplexities (↓) of models trained for 524B tokens on OWT. All perplexities for diffusion models are upper bounds.

	PTB	Wikitext	LM1B	Lambada	AG News	Pubmed	Arxiv
AR (Retrained)	**82.05**	**25.75**	**51.25**	51.28	**52.09**	49.01	41.73
SEDD (Retrained)	100.09	34.28	68.20	49.86	62.09	44.53	38.48
MDLM (Ours)	95.26	32.83	67.01	**47.52**	61.15	**41.89**	**37.37**

Table 4: GLUE evaluation results. Evaluation measures (↑) are F1 score for QQP and MRPC, Spearman correlations for STS-B, and accuracy for the rest. For MNLI, we report match/mismatch accuracies.

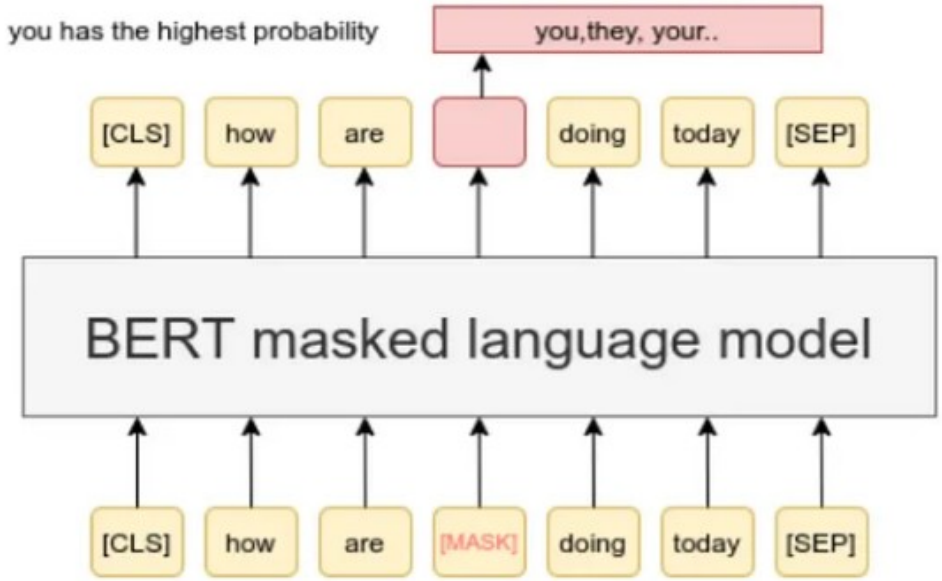
	MNLI (m/mm)	QQP	QNLI	SST-2	COLA	STS-B	MRPC	RTE	Avg
AR	80.94/80.78	86.98	86.16	90.14	33.43	84.32	83.88	47.29	74.88
BERT	84.43/85.35	88.41	**90.46**	**92.20**	54.81	**88.41**	89.16	61.37	81.62
+MDLM-FT	**84.76/85.07**	**88.49**	90.30	**92.20**	**57.69**	87.48	**90.53**	**62.09**	**82.06**

Is it just BERT trained differently?



Output

Input



- augment training by multiple degrees of “maskedness”
- simpler objective than previous generative BERT approaches